Delayed Job  for

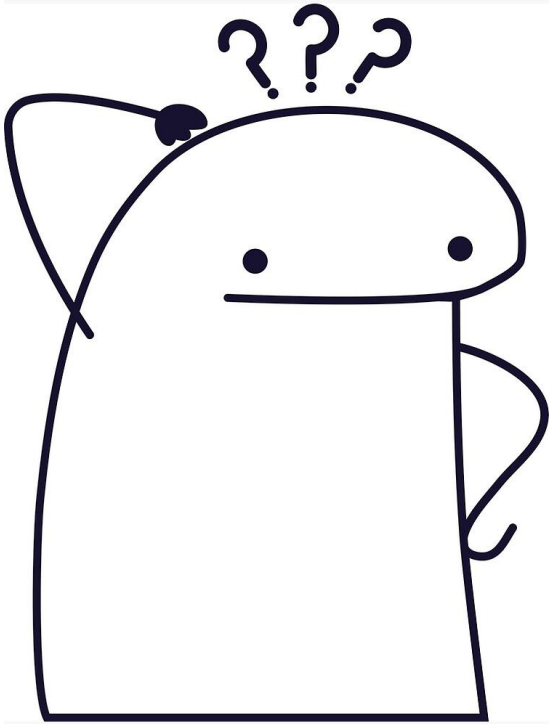
What even is it?

DelayedJob or for short is

- used for asynchronously executing longer tasks in the background
- a direct extraction from shopify where it is used for things like:
  - Sending newsletters
  - Batch imports
  - Spam checks
  - Etc.

How does it work?

- To use it with ActiveRecord we need the delayed_job and the delayed_job_activerecord gems
- After installing the gems we can execute the command rails g delayed_job:active_record
- This generates the migration for the jobs table, which is used by delayed_job to enqueue it’s jobs in the background

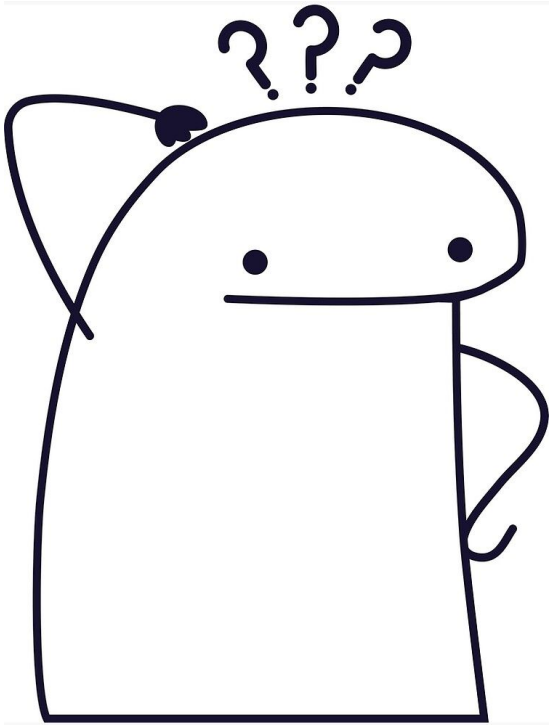


The delayed_job jobs table

```
create_table :delayed_jobs, :force => true do |table|
  table.integer  :priority, :default => 0      # Allows some jobs to jump to the front of the queue
  table.integer  :attempts, :default => 0      # Provides for retries, but still fail eventually.
  table.text     :handler                      # YAML-encoded string of the object that will do work
  table.text     :last_error                   # reason for last failure (See Note below)
  table.datetime :run_at                       # When to run. Could be Time.zone.now for immediately,
or sometime in the future.
  table.datetime :locked_at                    # Set when a client is working on this object
  table.datetime :failed_at                    # Set when all retries have failed (actually, by
default, the record is deleted instead)
  table.string   :locked_by                    # Who is working on this object (if locked)
  table.string   :queue                        # The name of the queue this job is in
  table.timestamps
end
```

How does it work?

- Additionally we need to tell ActiveJob to use delayed_job as queue adapter by setting the following config in the application.rb file: config.active_job.queue_adapter = :delayed_job
- After doing all this, the delayed_job worker can start working off the enqueued jobs



Use it!

- First off, run jobs:work to start working off the delayed jobs
- Call `.delay.method(params)` on any object and it will be processed in the background.

```
# without delayed_job
@user.activate!(@device)

# with delayed_job
@user.delay.activate!(@device)
```

- Delay() takes these parameters
  - :priority (number) lower numbers run first
  - :run_at (Time) run the job at this time
  - :queue (String) Named queue to put the job in

Delayed Cron Job

- We can use the delayed_cron_job gem to enqueue jobs that we want to execute repeatedly
- As the name says, the gem allows us to define a job with an assigned cron expression, which is then used to repeatedly execute the job

```ruby
class NightlyUpdatePeopleDataPtimeJob < CronJob
  self.cron_expression = '0 3 * * *'

  def perform(**args)
    if Skills.use_ptime_sync?
      Ptime::PeopleEmployees.new.update_people_data(**args)
    end
  end
end
```

Additional Tips and Tricks

- Additionally you can create a (empty) delayed_job model to access it on the rails console
- Crontab.guru is a useful website to check if your Cron expression works as expected

Links

https://github.com/collectiveidea/delayed_job

https://github.com/collectiveidea/delayed_job_active_record

https://github.com/codez/delayed_cron_job

https://crontab.guru/


THE SCHOOL PRESENTATION
THE PERSON WHO CREATED ALL THE SLIDES
THE PERSON WHO PUT EVERYTHING IN THE SLIDES
ME WHO JUST PUT MEMES ON THE LAST SLIDE THE WHOLE TIME
imgflip.com